SAAS GASTRONOMICO v1.0

Demo funcional - de cero a deploy

Doc	v1.0 - Paso a paso para arrancar la demo
Para	Renato (solo, antes de meter al equipo)
Stack	Next.js 14 + Supabase + Vercel
Costo	0 USD (todo free tier, salvo Claude Code)
Scope	Single-tenant (1 restaurante demo)
Resultado	URL publica con QR escaneable y flujo end-to-end
Tiempo	8-12 dias trabajando 4-6 hrs por dia
Fecha	28 de abril de 2026

Que vas a tener al final de este PDF: una URL publica donde podes generar un QR, pegarlo en una mesa, escanearlo con tu celular, ver el menu cargado por vos, armar un pedido, elegir efectivo (o ver placeholders de tarjeta/POS/VPOS/Google Pay), tipear RUC, confirmar, y ver como el pedido aparece en la pantalla de cocina/bar en menos de 1 segundo. Y desde el panel admin: editar menu, stock, mesas, zonas, generar QRs y ver stats basicas.

Que NO incluye esta v1.0: cobro real con Bancard (placeholder), facturacion SIFEN real (placeholder), multi-restaurante (single-tenant), app movil nativa, delivery con trackeo GPS, integraciones con Rappi/PedidosYa.

Indice

#	Bloque	Pag.
	TL;DR - resumen ejecutivo	3
	Diagrama 1: flujo end-to-end	4
	Diagrama 2: arquitectura tecnica	5
	Stack y costos	6
0	Pre-requisitos en tu compu	7
1	Crear cuentas (GitHub, Supabase, Vercel, Resend)	8
2	Inicializar repo Next.js + shadcn	9
3	Schema de base de datos (SQL completo)	10
4	RLS basica para single-tenant	13
5	Conexion Next.js + Supabase	14
6	Panel admin: mesas y zonas	15
7	Panel admin: menu, stock, perfil del local	17
8	Generador de QR por mesa	19
9	UI cliente post-QR (vista del menu)	20
10	Carrito y checkout (nombre, tel, RUC)	22
11	Pago (efectivo + placeholders tarjeta/VPOS/GPay)	23
12	Tickets por zona (cocina, bar, caja)	24
13	Realtime con Supabase	26
14	Seguimiento del pedido (cliente)	27
15	Estadisticas basicas en admin	28
16	Deploy a Vercel	29
17	Test end-to-end con QR real	30
A	Apendice: estructura de carpetas	31
B	Apendice: prompts utiles para Claude Code	32
C	Apendice: errores comunes y soluciones	33

TL;DR (resumen ejecutivo)

Vas a hacer 17 pasos. Cada paso te deja con algo funcionando que podes probar. No saltes pasos: el orden esta pensado para que cuando rompas algo, sepas que lo rompiste vos en el paso anterior, no que el sistema esta mal.

Resumen de los 17 pasos

Pasos	Que logras
0-1	Cuentas y herramientas instaladas. Listo para codear.
2	Repo Next.js corriendo en localhost:3000.
3-5	Supabase con tablas creadas y conectado a Next.js.
6-8	Panel admin funcional: mesas, zonas, menu, stock, QRs.
9-11	Cliente puede escanear QR, ver menu, armar pedido y 'pagar'.
12-14	Cocina/bar/caja ven tickets en realtime. Cliente ve estado.
15	Stats basicas: lo mas vendido, mesa mas usada.
16-17	Deploy en Vercel + prueba con QR real en una mesa.

Reglas no negociables mientras codeas

- Despues de cada paso: **git commit**. Si rompes algo en el paso 8, volves al 7 sin perder todo.
- Cada vez que tengas dudas, le mostras a Claude Code **el archivo entero**, no un fragmento. Tener menos contexto = mas alucinaciones.
- Si Claude Code te da un comando que no entendes, copialo en chat y le pedis que te lo explique **antes** de ejecutarlo.
- Siempre que cambies el schema de la BD, anota la fecha + razon en /docs/PROGRESS.md. Es 30 segundos y te salva en 3 meses.
- Variables sensibles (Supabase keys) NUNCA en el codigo: siempre en .env.local (gitignored) y en Vercel env vars.

Diagrama 1: flujo end-to-end

Lo que pasa desde que el cliente escanea el QR hasta que el ticket aparece en cocina.

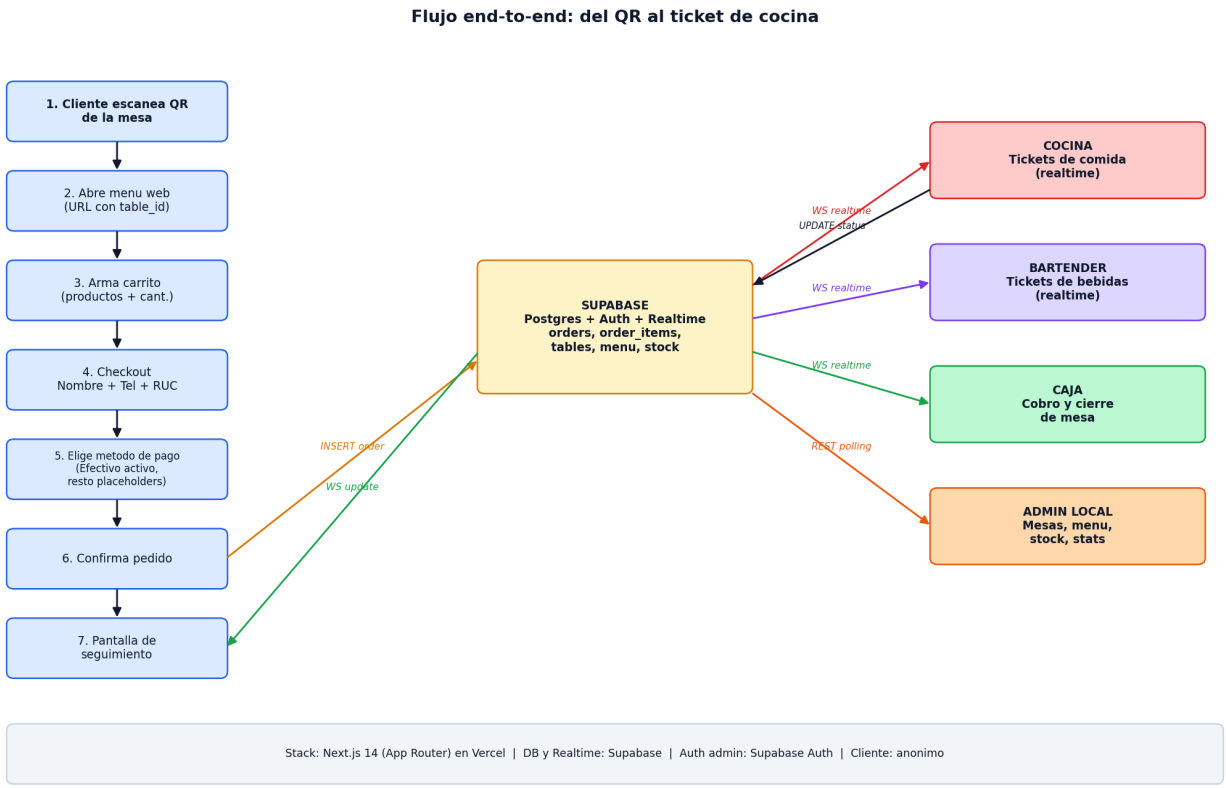
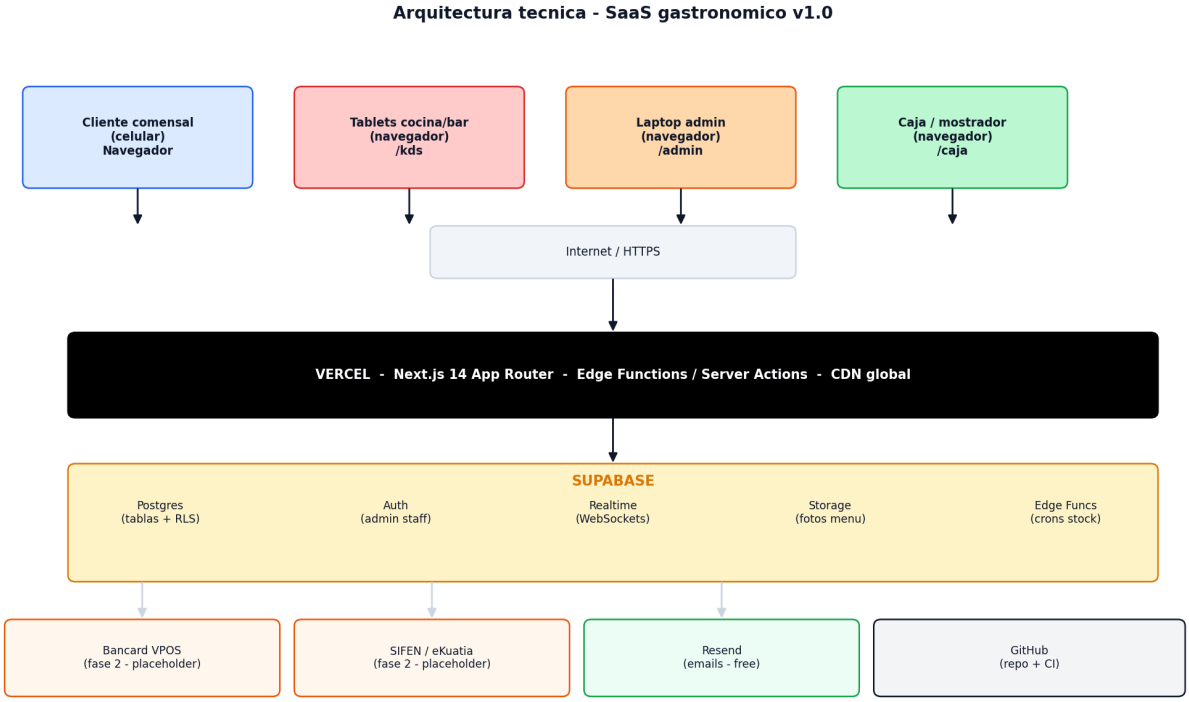


Diagrama 2: arquitectura tecnica

Como se comunican los pedazos. Todo lo que hay debajo de Vercel es servicio externo gratis.



Stack y costos

Lo que usas

Capa	Herramienta	Plan	Limite free
IDE	VS Code + Claude Code	Pago Claude Code	-
Repo	GitHub	Free	Repos privados ilimitados
Frontend / Backend	Next.js 14 (App Router)	OSS	-
UI lib	shadcn/ui + Tailwind	OSS	-
DB + Auth + Realtime	Supabase	Free	500MB DB, 2 proyectos, paused tras 7 dias inactivos
Hosting	Vercel	Hobby (Free)	100GB bandwidth/mes, no comercial
Email	Resend	Free	3.000 emails/mes, 100/dia
Storage	Supabase Storage	Free	1GB
Errores	Sentry	Free	5.000 events/mes
Uptime	BetterStack	Free	3 monitors, check cada 3 min
QR codes	qrcode.react (npm)	OSS	-
Iconos	lucide-react (npm)	OSS	-
Charts	recharts (npm)	OSS	-
Forms	react-hook-form + zod (npm)	OSS	-
State global	Zustand (npm)	OSS	-

Costo total v1.0

Cero USD. Todo dentro de free tiers. La unica plata sale de tu suscripcion a Claude Code.

Cuando vas a tener que pagar

Servicio	Trigger del upgrade	Costo USD/mes
Supabase Pro	DB > 500MB o necesitas branching o proyectos no-pause	25
Vercel Pro	Uso comercial real (TOS Hobby no permite produccion comercial)	20
Resend	Pasaste 3.000 emails/mes	20 desde 50k emails
Dominio .com.py	Anual (NIC.PY)	~30 USD/ano

OJO: Vercel Hobby NO permite uso comercial real. Para la demo y testing privado funciona, pero el dia que tengas un cliente pagando, debes pasar a Pro. Sino estas violando TOS.

Paso 0 - Pre-requisitos en tu compu

Antes de codear nada. Tiempo: 30 minutos.

0.1 Instalar lo que falta

Tool	Por que	Como verificar
Node.js 20+	Runtime de Next.js	node -v
pnpm	Package manager mas rapido que npm	pnpm -v
Git	Control de versiones	git --version
VS Code	Editor	Abrir y listo
Claude Code	Tu copiloto	claude --version
Supabase CLI	Para migraciones desde terminal	supabase --version
GitHub CLI	Para crear repo desde terminal	gh --version

0.2 Comandos de instalacion (Windows con PowerShell + winget)

```
# Si no tenes Node, lo instalas desde nodejs.org (LTS 20)
node -v # debe decir v20 o mayor

# pnpm:
npm install -g pnpm

# Supabase CLI:
winget install Supabase.cli
# o con scoop: scoop install supabase

# GitHub CLI:
winget install --id GitHub.cli

# Verificar todo:
node -v && pnpm -v && git --version && supabase --version && gh --version
```

0.3 Configurar Git

```
git config --global user.name "Renato"
git config --global user.email "tu@email.com"
git config --global init.defaultBranch main
```

0.4 Carpeta de trabajo

```
mkdir C:\Users\mancu\OneDrive\Desktop\ELMENUWEB\app
cd C:\Users\mancu\OneDrive\Desktop\ELMENUWEB\app
code .
```

OJO: OneDrive y Git pueden conflictuar (sync conflicts). Si te pasa, mueve la carpeta a C:\dev\saas-gastro y trabaja desde ahi.

Paso 1 - Crear cuentas y obtener API keys

Tiempo: 30 minutos. Hace todas estas con el mismo email.

1.1 GitHub

- Crear cuenta en github.com (si ya tenes, OK).
- Activar 2FA (Settings -> Password and authentication).
- Crear repo privado: gh repo create saas-gastro --private --source=. --remote=origin

1.2 Supabase

- supabase.com -> Sign in con GitHub.
- New project -> Name: saas-gastro-dev -> Region: South America (sao-paulo) -> Generar password fuerte y guardarlo en password manager.
- Esperar ~2 minutos a que provisione.
- Copiar de Settings -> API: Project URL, anon key, service_role key. Los necesitas despues.

1.3 Vercel

- vercel.com -> Sign up con GitHub.
- Authorizar acceso al repo saas-gastro (todavia no se conecta el proyecto, eso es despues).

1.4 Resend

- resend.com -> Sign up con GitHub.
- Settings -> API Keys -> Create -> guardar key.
- Para v1.0 no es critico (auth la cubre Supabase). Lo activas en Paso 5.

1.5 Guardar todas las keys en un .env.local (NO commitear)

```
# .env.local (en la raiz de tu repo, gitignored)
NEXT_PUBLIC_SUPABASE_URL=https://xxxxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=eyJhbGciOi...
SUPABASE_SERVICE_ROLE_KEY=eyJhbGciOi... # nunca expongas en cliente
RESEND_API_KEY=re_xxxxxx
NEXT_PUBLIC_APP_URL=http://localhost:3000 # cambia en prod
```

OJO: Si subis service_role_key a GitHub publico, te roban la base. Usar repo privado + revisar .gitignore antes del primer commit.

Paso 2 - Inicializar el repo Next.js + shadcn

Tiempo: 30 minutos. Resultado: localhost:3000 funcionando.

2.1 Crear el proyecto Next.js

```
cd C:\dev\saas-gastro # tu carpeta

pnpm create next-app@latest . --ts --app --tailwind --eslint --src-dir --import-alias "@/*"

# Responde:
# - TypeScript: Yes
# - ESLint: Yes
# - Tailwind: Yes
# - src/ directory: Yes
# - App Router: Yes
# - Turbopack: Yes
# - import alias: @/*
```

2.2 Instalar dependencias del proyecto

```
pnpm add @supabase/supabase-js @supabase/ssr
pnpm add zustand react-hook-form zod @hookform/resolvers
pnpm add lucide-react qrcode.react recharts
pnpm add date-fns
pnpm add -D @types/node
```

2.3 Inicializar shadcn/ui

```
pnpm dlx shadcn@latest init

# Responde:
# - Style: New York
# - Base color: Slate
# - CSS variables: Yes

# Agrega componentes que vas a usar:
pnpm dlx shadcn@latest add button input label card dialog form select \
table tabs toast dropdown-menu badge separator skeleton sheet alert
```

2.4 Probar que arranca

```
pnpm dev
# abrir http://localhost:3000
```

2.5 Primer commit

```
git add .
git commit -m "chore: bootstrap next.js + shadcn"
gh repo create saas-gastro --private --source=. --remote=origin --push
```

2.6 Crear estructura inicial de carpetas

Dentro de src/:

```
src/
app/
(admin)/ # rutas del panel admin (con layout propio)
admin/
page.tsx # dashboard
menu/
mesas/
stock/
stats/
perfil/
```

```
(cliente)/ # rutas del cliente comensal (sin auth)
m/[tableId]/ # menu post-QR
page.tsx
carrito/
checkout/
seguimiento/[orderId]/
(kds)/ # cocina, bar, caja
kds/
page.tsx # vista todo-en-uno
api/ # routes para webhooks externos (Bancard fase 2)
components/ # shadcn + propios
lib/
supabase/ # clients (server, browser)
types/ # types generados de Supabase
utils.ts
hooks/
```

Paso 3 - Schema de base de datos (SQL completo)

Tiempo: 1 hora. Lo pegas tal cual en Supabase SQL Editor.

3.1 Que tablas necesitas para v1.0

Tabla	Para que
restaurant_profile	Single-row con datos del local: nombre, logo, portada, tel, dir.
zones	Zonas del local: Salon, Terraza, Barra, etc.
tables	Mesas. FK a zone. Tiene un slug unico para el QR.
categories	Categorias del menu: Entradas, Pizzas, Bebidas, etc. Order.
station	Enum: cocina, bar, ninguna. Indica a que zona va el ticket.
products	Productos del menu. FK a category. Tiene station, foto, precio, stock.
product_modifiers	Variaciones: tamano, sin cebolla, etc (opcional v1.0 si te ahogas).
orders	Pedidos. FK a table. Status, total, payment_method, notes, ruc, customer_name, phone.
order_items	Renglones del pedido. FK a order y product. Qty, unit_price, station, item_status.
payments	Pagos asociados a orders. method, amount, status.
audit_log	Quien hizo que (lo bueno desde dia 1).

3.2 SQL para correr en Supabase (Editor SQL)

Pegar todo este bloque en Supabase Dashboard -> SQL Editor -> Run:

```
-- ==================================================
-- SCHEMA v1.0 - SaaS Gastronomico (single-tenant)
-- ==================================================

-- Extensiones
CREATE EXTENSION IF NOT EXISTS "pgcrypto";
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";

-- Enums
CREATE TYPE station_t AS ENUM ('cocina', 'bar', 'ninguna');
CREATE TYPE order_status_t AS ENUM ('pendiente', 'en_preparacion', 'listo', 'entregado',
'cobrado', 'cancelado');
CREATE TYPE item_status_t AS ENUM ('pendiente', 'en_preparacion', 'listo', 'entregado');
CREATE TYPE payment_method_t AS ENUM ('efectivo', 'tarjeta_pos', 'bancard_vpos', 'google_pay',
'transferencia');
CREATE TYPE payment_status_t AS ENUM ('pendiente', 'aprobado', 'rechazado');

-- Perfil del restaurante (single-row)
CREATE TABLE restaurant_profile (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
nombre text NOT NULL,
descripcion text,
logo_url text,
portada_url text,
telefono text,
direccion text,
```

```
ruc text,
color_primario text DEFAULT '#2563EB',
created_at timestamptz DEFAULT now(),
updated_at timestamptz DEFAULT now()
);

-- Zonas del local
CREATE TABLE zones (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
nombre text NOT NULL,
orden int DEFAULT 0,
created_at timestamptz DEFAULT now()
);

-- Mesas
CREATE TABLE tables (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
zone_id uuid REFERENCES zones(id) ON DELETE SET NULL,
numero text NOT NULL, -- 'M1', 'Barra-3'
slug text UNIQUE NOT NULL, -- token random para el QR
capacidad int DEFAULT 4,
activa boolean DEFAULT true,
created_at timestamptz DEFAULT now()
);
CREATE INDEX ON tables(zone_id);
CREATE INDEX ON tables(slug);

-- Categorias del menu
CREATE TABLE categories (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
nombre text NOT NULL,
orden int DEFAULT 0,
activa boolean DEFAULT true
);

-- Productos
CREATE TABLE products (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
category_id uuid REFERENCES categories(id) ON DELETE SET NULL,
nombre text NOT NULL,
descripcion text,
foto_url text,
precio numeric(12,0) NOT NULL, -- PYG sin decimales
station station_t NOT NULL DEFAULT 'cocina',
stock_disponible int, -- null = stock infinito
activo boolean DEFAULT true,
created_at timestamptz DEFAULT now(),
updated_at timestamptz DEFAULT now()
);
CREATE INDEX ON products(category_id);
CREATE INDEX ON products(station);

-- Pedidos
CREATE TABLE orders (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
table_id uuid REFERENCES tables(id),
numero_visible serial, -- numero correlativo para mostrar
customer_name text,
customer_phone text,
ruc text, -- '80012345-6' o null si no factura
total numeric(12,0) NOT NULL DEFAULT 0,
status order_status_t DEFAULT 'pendiente',
payment_method payment_method_t,
notas text, -- 'sin cebolla en la 2'
```

```
created_at timestamptz DEFAULT now(),
updated_at timestamptz DEFAULT now()
);
CREATE INDEX ON orders(table_id);
CREATE INDEX ON orders(status);
CREATE INDEX ON orders(created_at DESC);

-- Items del pedido
CREATE TABLE order_items (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
order_id uuid REFERENCES orders(id) ON DELETE CASCADE NOT NULL,
product_id uuid REFERENCES products(id),
product_nombre_snapshot text NOT NULL, -- para que si despues editas el producto, el ticket no
cambia
qty int NOT NULL CHECK (qty > 0),
unit_price numeric(12,0) NOT NULL,
station station_t NOT NULL,
item_status item_status_t DEFAULT 'pendiente',
notas text,
created_at timestamptz DEFAULT now()
);
CREATE INDEX ON order_items(order_id);
CREATE INDEX ON order_items(station, item_status);

-- Pagos
CREATE TABLE payments (
id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
order_id uuid REFERENCES orders(id) ON DELETE CASCADE,
method payment_method_t NOT NULL,
amount numeric(12,0) NOT NULL,
status payment_status_t DEFAULT 'pendiente',
external_ref text, -- merchant_ref de Bancard, etc
created_at timestamptz DEFAULT now()
);

-- Audit log basico
CREATE TABLE audit_log (
id bigserial PRIMARY KEY,
occurred_at timestamptz DEFAULT now(),
actor text,
action text NOT NULL,
resource_type text,
resource_id text,
meta jsonb
);

-- Funcion para descontar stock al crear order_item
CREATE OR REPLACE FUNCTION descontar_stock() RETURNS TRIGGER AS $$
BEGIN
UPDATE products
SET stock_disponible = GREATEST(stock_disponible - NEW.qty, 0),
updated_at = now()
WHERE id = NEW.product_id
AND stock_disponible IS NOT NULL;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_descontar_stock
AFTER INSERT ON order_items
FOR EACH ROW EXECUTE FUNCTION descontar_stock();

-- Funcion para recalcular total del order
```

```
CREATE OR REPLACE FUNCTION recalcular_total() RETURNS TRIGGER AS $$
BEGIN
UPDATE orders SET total = (
SELECT COALESCE(SUM(qty * unit_price), 0)
FROM order_items WHERE order_id = NEW.order_id
) WHERE id = NEW.order_id;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_recalcular_total
AFTER INSERT OR UPDATE OR DELETE ON order_items
FOR EACH ROW EXECUTE FUNCTION recalcular_total();
```

3.3 Insertar datos iniciales (seed minimo)

```
INSERT INTO restaurant_profile (nombre, telefono, direccion, color_primario)
VALUES ('Mi Restaurante Demo', '+595 981 000000', 'Asuncion, Paraguay', '#2563EB');

INSERT INTO zones (nombre, orden) VALUES
('Salon', 1), ('Terraza', 2), ('Barra', 3);

INSERT INTO categories (nombre, orden) VALUES
('Entradas', 1), ('Principales', 2), ('Pizzas', 3),
('Postres', 4), ('Bebidas', 5), ('Cocteles', 6);
```

Paso 4 - RLS basica para single-tenant

RLS (Row Level Security): politicas en Postgres que filtran filas por usuario. En single-tenant tu RLS es simple: cliente anonimo puede leer menu publico y crear orders, staff autenticado puede leer/escribir todo.

4.1 Habilitar RLS en cada tabla

```
ALTER TABLE restaurant_profile ENABLE ROW LEVEL SECURITY;
ALTER TABLE zones ENABLE ROW LEVEL SECURITY;
ALTER TABLE tables ENABLE ROW LEVEL SECURITY;
ALTER TABLE categories ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE order_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE payments ENABLE ROW LEVEL SECURITY;
ALTER TABLE audit_log ENABLE ROW LEVEL SECURITY;
```

4.2 Politicas mas simples para arrancar

```
-- Lectura publica del menu y mesas (anyone)
CREATE POLICY "menu publico" ON products FOR SELECT USING (activo = true);
CREATE POLICY "categorias publicas" ON categories FOR SELECT USING (activa = true);
CREATE POLICY "zonas publicas" ON zones FOR SELECT USING (true);
CREATE POLICY "mesas publicas" ON tables FOR SELECT USING (activa = true);
CREATE POLICY "perfil publico" ON restaurant_profile FOR SELECT USING (true);

-- Cliente anonimo puede crear orders e items
CREATE POLICY "anyone crea order" ON orders FOR INSERT WITH CHECK (true);
CREATE POLICY "anyone crea items" ON order_items FOR INSERT WITH CHECK (true);
CREATE POLICY "anyone crea payment" ON payments FOR INSERT WITH CHECK (true);

-- Cliente puede leer SU pedido (por ahora todos pueden leer todos -- en v2 lo restringimos por
session/cookie)
CREATE POLICY "anyone lee orders" ON orders FOR SELECT USING (true);
CREATE POLICY "anyone lee items" ON order_items FOR SELECT USING (true);
CREATE POLICY "anyone lee payments" ON payments FOR SELECT USING (true);

-- Staff autenticado: puede todo
CREATE POLICY "staff escribe perfil" ON restaurant_profile FOR ALL
USING (auth.role() = 'authenticated') WITH CHECK (auth.role() = 'authenticated');
CREATE POLICY "staff escribe zones" ON zones FOR ALL
USING (auth.role() = 'authenticated') WITH CHECK (auth.role() = 'authenticated');
-- repetir para tables, categories, products, orders update, order_items update, payments
update, audit_log
```

OJO: Estas politicas son INTENCIONALMENTE simples para v1.0. En v2.0 vas a restringir lectura del pedido por session token (cookie) para que un cliente no pueda leer pedidos de otro cliente con solo cambiar el ID en la URL.

Paso 5 - Conexion Next.js + Supabase

Tiempo: 30 minutos. Tres archivos.

5.1 Cliente Supabase para el browser

```
// src/lib/supabase/client.ts
import { createBrowserClient } from '@supabase/ssr'

export function createClient() {
return createBrowserClient(
process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
)
}
```

5.2 Cliente Supabase para Server Components

```
// src/lib/supabase/server.ts
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'

export async function createClient() {
const cookieStore = await cookies()
return createServerClient(
process.env.NEXT_PUBLIC_SUPABASE_URL!,
process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
{
cookies: {
getAll() { return cookieStore.getAll() },
setAll(list) { list.forEach(({ name, value, options }) => cookieStore.set(name, value,
options)) },
},
}
)
}
```

5.3 Generar tipos TypeScript de tu schema

```
supabase login
supabase link --project-ref TU_PROJECT_REF
supabase gen types typescript --linked > src/lib/types/database.ts
```

Cada vez que cambies el schema, corres el ultimo comando otra vez.

5.4 Probar la conexion

```
// src/app/page.tsx (provisorio para testear)
import { createClient } from '@/lib/supabase/server'

export default async function Home() {
const supabase = await createClient()
const { data: zones } = await supabase.from('zones').select('*')
return (
<main className='p-8'>
<h1 className='text-2xl font-bold'>Zonas:</h1>
<ul>{zones?.map(z => <li key={z.id}>{z.nombre}</li>)}</ul>
</main>
)
}
```

Si ves Salon/Terraza/Barra: estas conectado. Commit.

Paso 6 - Panel admin: mesas y zonas

Tiempo: 1 dia. Resultado: podes crear/editar/borrar zonas y mesas.

6.1 Auth del admin

Para el demo, creas un solo usuario admin en Supabase Dashboard -> Authentication -> Add user (con tu email + password). En v1.0 no haces signup ni invitations, solo login. Te logueas y entras a /admin.

6.2 Layout protegido

```
// src/app/(admin)/admin/layout.tsx
import { createClient } from '@/lib/supabase/server'
import { redirect } from 'next/navigation'
import { AdminSidebar } from '@/components/admin/sidebar'

export default async function AdminLayout({ children }: { children: React.ReactNode }) {
const supabase = await createClient()
const { data: { user } } = await supabase.auth.getUser()
if (!user) redirect('/login')
return (
<div className='flex min-h-screen'>
<AdminSidebar />
<main className='flex-1 p-8'>{children}</main>
</div>
)
}
```

6.3 Pagina /admin/mesas

Funcionalidad minima: lista de zonas con sus mesas adentro, boton 'agregar zona', boton 'agregar mesa' (eligiendo a que zona pertenece), boton editar/borrar en cada uno. La creacion de mesa genera automaticamente el slug (token random) que sera la URL del QR.

```
// Helper para generar slug random
// src/lib/utils.ts
export function generateSlug(): string {
// 10 chars, sin caracteres confusos
const chars = 'abcdefghjkmnpqrstuvwxyz23456789'
return Array.from({ length: 10 }, () =>
chars[Math.floor(Math.random() * chars.length)]
).join('')
}
```

6.4 Server Action para crear zona

```
// src/app/(admin)/admin/mesas/actions.ts
'use server'
import { createClient } from '@/lib/supabase/server'
import { revalidatePath } from 'next/cache'

export async function crearZona(formData: FormData) {
const supabase = await createClient()
const nombre = formData.get('nombre') as string
const { error } = await supabase.from('zones').insert({ nombre })
if (error) throw error
revalidatePath('/admin/mesas')
}
```

6.5 Lo mismo para mesas

Crear server actions crearMesa, editarMesa, borrarMesa, toggleActiva. Mismo patron.

6.6 Pedile a Claude Code

```
Tarea: implementa /admin/mesas

Funcionalidad:
1. Lista las zonas con sus mesas usando server component
2. Boton 'Nueva zona' que abre Dialog con form (react-hook-form + zod)
3. Por cada zona, boton 'Nueva mesa' que abre Dialog con form
(campos: numero, capacidad)
4. Cada mesa tiene un dropdown con: editar, ver QR, desactivar, eliminar
5. Usar shadcn/ui (Card, Dialog, Form, Input, Button, DropdownMenu)
6. Usar Server Actions de Next.js 14 para mutaciones
7. Validar con zod: nombre min 1 max 50, numero unico por zona

Archivos involucrados:
- src/app/(admin)/admin/mesas/page.tsx (server component, lista)
- src/app/(admin)/admin/mesas/actions.ts (server actions)
- src/components/admin/zona-card.tsx
- src/components/admin/nueva-zona-dialog.tsx
- src/components/admin/nueva-mesa-dialog.tsx
```

Paso 7 - Panel admin: menu, stock, perfil

Tiempo: 2 dias. Resultado: podes gestionar todo el menu del local.

7.1 /admin/menu

- Lista de categorias (orden, activa) con CRUD.
- Por cada categoria, lista de productos.
- Crear/editar producto con: nombre, descripcion, precio, foto (Supabase Storage), station (cocina/bar/ninguna), stock_disponible, activo.
- Drag and drop para reordenar productos dentro de la categoria.

7.2 Subir foto a Supabase Storage

```
-- Crear bucket en Supabase Dashboard -> Storage -> New bucket
-- Name: 'productos'
-- Public: yes (para que las URLs sean accesibles)

-- Politica de upload (autenticado):
CREATE POLICY "auth uploads productos" ON storage.objects
FOR INSERT TO authenticated
WITH CHECK (bucket_id = 'productos');

CREATE POLICY "public read productos" ON storage.objects
FOR SELECT TO public
USING (bucket_id = 'productos');
```

```
// Componente que sube foto y guarda URL
// src/components/admin/foto-uploader.tsx
'use client'
import { createClient } from '@/lib/supabase/client'

export function FotoUploader({ onUpload }: { onUpload: (url: string) =&gt; void }) {
const supabase = createClient()
async function handle(e: React.ChangeEvent&lt;HTMLInputElement&gt;) {
const file = e.target.files?.[0]
if (!file) return
const path = `${Date.now()}-${file.name}`
const { data, error } = await supabase.storage.from('productos').upload(path, file)
if (error) { alert(error.message); return }
const { data: pub } = supabase.storage.from('productos').getPublicUrl(path)
onUpload(pub.publicUrl)
}
return &lt;input type='file' accept='image/*' onChange={handle} /&gt;
}
```

7.3 /admin/stock

Vista tabla con todos los productos que tengan stock_disponible distinto de null. Columnas: foto, nombre, stock actual, ultima venta, boton 'reponer' que abre modal para sumar cantidad.

El descuento automatico ya lo hace el trigger SQL del Paso 3 (descontar_stock). Vos solo necesitas la UI para reponer.

```
// Server action para reponer stock
export async function reponerStock(productId: string, cantidad: number) {
const supabase = await createClient()
const { data: prod } = await supabase
.from('products').select('stock_disponible').eq('id', productId).single()
const nuevo = (prod?.stock_disponible ?? 0) + cantidad
await supabase.from('products')
.update({ stock_disponible: nuevo, updated_at: new Date().toISOString() })
```

```
.eq('id', productId)
await supabase.from('audit_log').insert({
action: 'stock.reponer',
resource_type: 'product',
resource_id: productId,
meta: { cantidad, nuevo_stock: nuevo },
})
revalidatePath('/admin/stock')
}
```

7.4 /admin/perfil

Form con campos del restaurant_profile: nombre, descripcion, telefono, direccion, RUC, color primario (color picker), foto de perfil (logo) y portada (subidos a Storage).

Usar shadcn Form + react-hook-form + zod para validar.

7.5 Pedile a Claude Code

```
Tarea: implementar /admin/menu, /admin/stock, /admin/perfil

Especificaciones:
1. /admin/menu: tabs con categorias. CRUD productos por tab.
2. /admin/stock: tabla shadcn con productos que tienen stock numerico.
Columna stock con badge rojo si menor a 5.
3. /admin/perfil: formulario con campos del restaurant_profile.
4. Usar Supabase Storage bucket 'productos' para fotos.
5. Validacion zod estricta. Errores inline en el form.
6. Toast (sonner via shadcn) en exito/error.

Archivos: page.tsx + actions.ts en cada ruta + componentes en /components/admin/
```

Paso 8 - Generador de QR por mesa

Tiempo: 2 horas.

8.1 La URL que codifica el QR

```
https://tudominio.com/m/{slug_de_la_mesa}
```

El slug es el token random que generaste al crear la mesa. La URL es publica, sin auth.

8.2 Componente para mostrar/imprimir QR

```
// src/components/admin/qr-mesa.tsx
'use client'
import { QRCodeSVG } from 'qrcode.react'

export function QrMesa({ slug, numero }: { slug: string; numero: string }) {
const url = `${process.env.NEXT_PUBLIC_APP_URL}/m/${slug}`
return (
<div className='flex flex-col items-center gap-2 p-4 border rounded'>
<QRCodeSVG value={url} size={200} includeMargin />
<p className='font-bold'>{numero}</p>
<p className='text-xs text-gray-500'>{url}</p>
</div>
)
}
```

8.3 Pagina /admin/mesas/qrs (imprimir todos)

Vista que muestra todos los QRs activos en grid 3x3 con un boton 'Imprimir'. Usa CSS @media print para esconder navegacion al imprimir.

```
// src/app/(admin)/admin/mesas/qrs/page.tsx
import { createClient } from '@/lib/supabase/server'
import { QrMesa } from '@/components/admin/qr-mesa'

export default async function QrsPage() {
const supabase = await createClient()
const { data: mesas } = await supabase
.from('tables').select('id, numero, slug').eq('activa', true)
.order('numero')
return (
<div className='p-8'>
<button onClick={() => window.print()} className='print:hidden mb-4'>Imprimir</button>
<div className='grid grid-cols-3 gap-4'>
{mesas?.map(m => <QrMesa key={m.id} slug={m.slug} numero={m.numero} />)}
</div>
</div>
)
}
```

OJO: El boton onClick en server component no funciona. Hay que envolver en client component o usar un Link a window.print(). Claude Code te lo arregla.

Paso 9 - UI cliente post-QR (vista del menu)

Tiempo: 2 dias. Resultado: el cliente escanea, ve el menu, agrega productos.

9.1 Ruta dinamica /m/[slug]

```
// src/app/(cliente)/m/[slug]/page.tsx
import { createClient } from '@/lib/supabase/server'
import { notFound } from 'next/navigation'
import { MenuView } from '@/components/cliente/menu-view'

export default async function MenuPage({ params }: { params: Promise&lt;{ slug: string }&gt; })
{
const { slug } = await params
const supabase = await createClient()

const { data: mesa } = await supabase
.from('tables').select('id, numero, zone_id, zones(nombre)')
.eq('slug', slug).single()

if (!mesa) notFound()

const { data: profile } = await supabase.from('restaurant_profile').select('*').single()
const { data: categorias } = await supabase
.from('categories').select('*').eq('activa', true).order('orden')
const { data: productos } = await supabase
.from('products').select('*').eq('activo', true)
.or('stock_disponible.is.null,stock_disponible.gt.0')

return &lt;MenuView mesa={mesa} profile={profile} categorias={categorias}
productos={productos} /&gt;
}
```

9.2 Componente MenuView (client)

Maneja: listado por categorias, busqueda, badge 'En tu mesa' arriba, carrito flotante al pie.

```
// src/components/cliente/menu-view.tsx
'use client'
import { useCarrito } from '@/hooks/use-carrito'

export function MenuView({ mesa, profile, categorias, productos }: any) {
const { items, agregar } = useCarrito()
return (
<div className='min-h-screen bg-gray-50'>
<header className='relative h-40 bg-cover bg-center'
style={{ backgroundImage: `url(${profile.portada_url})` }}>
<div className='absolute inset-0 bg-black/40' />
<div className='absolute bottom-4 left-4 text-white'>
<h1 className='text-2xl font-bold'>{profile.nombre}</h1>
<p className='text-sm'>Mesa {mesa.numero} - {mesa.zones?.nombre}</p>
</div>
</header>
<main className='p-4 pb-24'>
{categorias.map((cat: any) => (
<section key={cat.id} className='mb-8'>
<h2 className='text-lg font-bold mb-3'>{cat.nombre}</h2>
<div className='grid gap-3'>
{productos.filter((p: any) => p.category_id === cat.id).map((p: any) => (
<ProductoCard key={p.id} producto={p} onAdd={() => agregar(p)} />
))}
</div>
</section>
))}
```

```
</main>
{items.length > 0 && <CarritoFlotante mesa={mesa} />}
</div>
)
}
```

9.3 Estado del carrito (Zustand)

```
// src/hooks/use-carrito.ts
import { create } from 'zustand'
import { persist } from 'zustand/middleware'

type Item = { id: string; nombre: string; precio: number; qty: number; station: string }

type CarritoStore = {
items: Item[]
agregar: (p: any) => void
cambiarQty: (id: string, qty: number) => void
remover: (id: string) => void
limpiar: () => void
total: () => number
}

export const useCarrito = create<CarritoStore>()(
persist(
(set, get) => ({
items: [],
agregar: (p) => set(s => {
const exist = s.items.find(i => i.id === p.id)
if (exist) return { items: s.items.map(i => i.id === p.id ? { ...i, qty: i.qty + 1 } : i) }
return { items: [...s.items, { id: p.id, nombre: p.nombre, precio: p.precio, qty: 1, station:
p.station }] }
}),
cambiarQty: (id, qty) => set(s => ({ items: s.items.map(i => i.id === id ? { ...i, qty } : i)
})),
remover: (id) => set(s => ({ items: s.items.filter(i => i.id !== id) })),
limpiar: () => set({ items: [] }),
total: () => get().items.reduce((acc, i) => acc + i.precio * i.qty, 0),
}),
{ name: 'carrito' }
)
)
```

Paso 10 - Carrito y checkout

Tiempo: 1 dia.

10.1 Pagina /m/[slug]/carrito

Lista de items con boton +/- para cambiar qty, total abajo, boton 'Continuar al checkout'.

10.2 Pagina /m/[slug]/checkout

Form con:

- Nombre (obligatorio, min 2).
- Telefono (obligatorio, formato PY +595 o local).
- RUC (opcional, formato 80012345-6 con regex).
- Notas para la cocina (opcional).
- Seleccion de metodo de pago (Paso 11).

10.3 Validacion zod

```
// src/lib/schemas/checkout.ts
import { z } from 'zod'

export const checkoutSchema = z.object({
nombre: z.string().min(2, 'Mínimo 2 caracteres').max(50),
telefono: z.string().min(8, 'Telefono invalido').max(20),
ruc: z.string().regex(/^[0-9]{6,8}-[0-9]$/, 'Formato:
80012345-6').optional().or(z.literal('')),
notas: z.string().max(200).optional(),
payment_method: z.enum(['efectivo', 'tarjeta_pos', 'bancard_vpos', 'google_pay',
'transferencia']),
})
export type CheckoutInput = z.infer&lt;typeof checkoutSchema&gt;
```

10.4 Server action para crear el pedido

```
// src/app/(cliente)/m/[slug]/checkout/actions.ts
'use server'
import { createClient } from '@/lib/supabase/server'
import { checkoutSchema } from '@/lib/schemas/checkout'
import { redirect } from 'next/navigation'

export async function crearPedido(slug: string, items: any[], data: any) {
const parsed = checkoutSchema.parse(data)
const supabase = await createClient()

const { data: mesa } = await supabase.from('tables').select('id').eq('slug', slug).single()
if (!mesa) throw new Error('Mesa no encontrada')

const { data: order, error } = await supabase
.from('orders')
.insert({
table_id: mesa.id,
customer_name: parsed.nombre,
customer_phone: parsed.telefono,
ruc: parsed.ruc || null,
notas: parsed.notas,
payment_method: parsed.payment_method,
status: 'pendiente',
})
.select().single()
if (error || !order) throw error

await supabase.from('order_items').insert(
items.map(i => ({
order_id: order.id,
product_id: i.id,
product_nombre_snapshot: i.nombre,
qty: i.qty,
unit_price: i.precio,
station: i.station,
}))
)
await supabase.from('payments').insert({
order_id: order.id,
method: parsed.payment_method,
amount: items.reduce((acc, i) =&gt; acc + i.precio * i.qty, 0),
status: parsed.payment_method === 'efectivo' ? 'pendiente' : 'pendiente',
})
redirect(`/m/${slug}/seguimiento/${order.id}`)
}
```

Paso 11 - Pago: efectivo + placeholders

Tiempo: 4 horas.

11.1 UI de seleccion de metodo

Mostras los 5 metodos como Cards con icono. Solo *efectivo* hace algo real (marca el pedido como pendiente de cobro en caja). Los otros 4 muestran un modal: 'Proximamente. Por ahora confirma el pedido y pagas en caja'.

```
// src/components/cliente/payment-selector.tsx
const METODOS = [
{ value: 'efectivo', label: 'Efectivo', icon: '■', activo: true },
{ value: 'tarjeta_pos', label: 'Tarjeta (POS en mesa)', icon: '■', activo: false },
{ value: 'bancard_vpos', label: 'Tarjeta online (Bancard)', icon: '■', activo: false },
{ value: 'google_pay', label: 'Google Pay', icon: 'G', activo: false },
{ value: 'transferencia', label: 'Transferencia', icon: '■', activo: false },
]
```

OJO: Los emojis los podes usar en JSX/HTML pero NO en este PDF (limitacion de la fuente). En el codigo real Tailwind + lucide-react son mejores que emojis (mas profesional).

11.2 Por que dejar los placeholders ya

- Cliente ve que el sistema 'soporta' multiples metodos -> percepcion de producto serio.
- Cuando integres Bancard real, no haces refactor de UI, solo cableas el flujo.
- Sirve para entrevistas comerciales: 'lo armamos para soportar todo desde el principio'.

11.3 Como va a ser cuando integres Bancard VPOS (preview)

Bancard VPOS es 3DS (3D Secure). Flujo: tu backend crea un token con la API de Bancard, redirigis al cliente al iframe de Bancard, cliente carga tarjeta, Bancard te postea un webhook cuando aprueba/rechaza, vos actualizas payments.status.

```
// Estructura sugerida (fase 2):
// src/lib/bancard/
// client.ts // wrapper de la API
// types.ts
// src/app/api/webhooks/bancard/route.ts // recibe confirmaciones

// Endpoint que crea el token (fase 2):
// POST /api/payments/bancard/create
// body: { order_id }
// resp: { iframe_url, token }
```

Paso 12 - Tickets por zona (cocina, bar, caja)

Tiempo: 1 dia. Como dijiste, una sola pantalla con tickets distintos por zona (separados visualmente). Ruta: /kds (Kitchen Display System).

12.1 Layout

```
+-------------------------------------------------------------+
| KDS - Vista general [Cocina][Bar][Caja] |
+-------------------------------------------------------------+
| COCINA (3 tickets) BAR (1) CAJA (2) |
| +-------+ +-------+ +-------+ +-------+ |
| | M1 | | M3 | | M2 | | M5 | |
| | 12:34 | | 12:36 | | 12:35 | | $45k | |
| | 2x ham| | 1x emp| | 2x cer| | EFECT | |
| | 1x pap| | | | | | | |
| | [LIST]| | [LIST]| | [LIST]| | [COBR]| |
| +-------+ +-------+ +-------+ +-------+ |
+-------------------------------------------------------------+
```

12.2 Logica

- **Cocina**: muestra order_items con station=cocina y status pendiente o en_preparacion. Boton 'Listo' marca item_status=listo.
- **Bar**: idem pero station=bar.
- **Caja**: muestra orders donde TODOS los items estan en estado listo o entregado. Boton 'Cobrar' marca order.status=cobrado.
- Cuando todos los items de un order estan listos, el cliente recibe notificacion 'pedido listo' (paso 14).

12.3 Query principal del KDS

```
// src/app/(kds)/kds/page.tsx
import { createClient } from '@/lib/supabase/server'
import { KdsView } from '@/components/kds/kds-view'

export default async function KdsPage() {
const supabase = await createClient()
const { data: items } = await supabase
.from('order_items')
.select(`
id, qty, product_nombre_snapshot, station, item_status, notas, created_at,
orders!inner(id, numero_visible, table_id, status, tables(numero))
`)
.in('item_status', ['pendiente', 'en_preparacion'])
.order('created_at')

return &lt;KdsView initialItems={items ?? []} /&gt;
}
```

12.4 Server action para marcar listo

```
// src/app/(kds)/kds/actions.ts
'use server'
import { createClient } from '@/lib/supabase/server'
import { revalidatePath } from 'next/cache'

export async function marcarItemListo(itemId: string) {
const supabase = await createClient()
await supabase.from('order_items').update({ item_status: 'listo' }).eq('id', itemId)
revalidatePath('/kds')
}

export async function cobrarOrder(orderId: string) {
const supabase = await createClient()
await supabase.from('orders').update({ status: 'cobrado' }).eq('id', orderId)
await supabase.from('payments').update({ status: 'aprobado' }).eq('order_id', orderId)
revalidatePath('/kds')
}
```

Paso 13 - Realtime con Supabase

Tiempo: 4 horas. Resultado: cuando el cliente confirma un pedido, aparece en cocina sin refresh.

13.1 Habilitar Realtime en las tablas

En Supabase Dashboard -> Database -> Replication -> activar:

- orders
- order_items

13.2 Subscribirse desde el componente KDS

```
// src/components/kds/kds-view.tsx
'use client'
import { useEffect, useState } from 'react'
import { createClient } from '@/lib/supabase/client'

export function KdsView({ initialItems }: { initialItems: any[] }) {
const [items, setItems] = useState(initialItems)
const supabase = createClient()

useEffect(() => {
const channel = supabase.channel('kds-items')
.on('postgres_changes',
{ event: 'INSERT', schema: 'public', table: 'order_items' },
(payload) => setItems(curr => [...curr, payload.new])
)
.on('postgres_changes',
{ event: 'UPDATE', schema: 'public', table: 'order_items' },
(payload) => setItems(curr =>
curr.map(i => i.id === payload.new.id ? { ...i, ...payload.new } : i)
.filter(i => ['pendiente', 'en_preparacion'].includes(i.item_status))
)
)
.subscribe()
return () => { supabase.removeChannel(channel) }
}, [supabase])

// ... render por station ...
}
```

OJO: El payload del INSERT viene SIN los joins del query original. Si necesitas el numero de mesa, lo buscas con un fetch adicional o lo metes en el INSERT inicial. Lo simple para v1.0: cada vez que llega INSERT, refrescas todo el listado con un fetch.

Paso 14 - Seguimiento del pedido (cliente)

Tiempo: 4 horas.

14.1 Pagina /m/[slug]/seguimiento/[orderId]

Muestra estados visualmente con timeline:

```
Pedido recibido [✓] 12:34
En preparacion [✓] 12:36
Listo para entregar [ ] esperando...
Entregado [ ]
Cobrado [ ]
```

14.2 Logica de estados

Estado del order	Cuando se setea
pendiente	Al crear el pedido.
en_preparacion	Cuando algun item pasa a en_preparacion (manual o auto al primer 'tomar').
listo	Cuando TODOS los items estan listos (trigger).
entregado	Cuando mozo marca 'entregue' en la mesa (boton en KDS).
cobrado	Cuando caja marca pagado.

14.3 Trigger para auto-pasar a 'listo' cuando todos los items estan listos

```
CREATE OR REPLACE FUNCTION check_order_listo() RETURNS TRIGGER AS $$
DECLARE
pendientes int;
BEGIN
SELECT COUNT(*) INTO pendientes
FROM order_items
WHERE order_id = NEW.order_id
AND item_status NOT IN ('listo', 'entregado');
IF pendientes = 0 THEN
UPDATE orders SET status = 'listo'
WHERE id = NEW.order_id AND status NOT IN ('cobrado', 'cancelado');
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_check_order_listo
AFTER UPDATE OF item_status ON order_items
FOR EACH ROW EXECUTE FUNCTION check_order_listo();
```

14.4 Suscripcion realtime en seguimiento

Mismo patron que en KDS pero suscribis a un solo order_id y a sus order_items. Cada update redibuja el timeline.

Paso 15 - Estadisticas basicas en admin

Tiempo: 1 dia. Solo 4 stats para v1.0:

Metrica	Query
Pedidos hoy	COUNT(*) WHERE created_at::date = CURRENT_DATE
Facturado hoy	SUM(total) WHERE status='cobrado' AND created_at::date = CURRENT_DATE
Producto mas vendido (7 dias)	GROUP BY product_id ORDER BY SUM(qty) DESC LIMIT 1
Mesa mas usada (7 dias)	GROUP BY table_id ORDER BY COUNT(orders) DESC LIMIT 1

15.1 Vista SQL para reusar

```
CREATE VIEW v_stats_diarias AS
SELECT
DATE(created_at) AS fecha,
COUNT(*) AS pedidos,
SUM(total) FILTER (WHERE status = 'cobrado') AS facturado,
COUNT(DISTINCT table_id) AS mesas_usadas
FROM orders
GROUP BY DATE(created_at);

CREATE VIEW v_top_productos AS
SELECT
oi.product_nombre_snapshot,
oi.product_id,
SUM(oi.qty) AS total_vendido,
SUM(oi.qty * oi.unit_price) AS facturado
FROM order_items oi
JOIN orders o ON o.id = oi.order_id
WHERE o.status = 'cobrado'
GROUP BY oi.product_id, oi.product_nombre_snapshot;
```

15.2 Pagina /admin/stats

Cards arriba con los 4 KPIs. Abajo, grafico de barras (recharts) con pedidos por dia ultimos 14 dias. Tabla con top 10 productos.

Paso 16 - Deploy a Vercel

Tiempo: 1 hora.

16.1 Conectar repo a Vercel

- vercel.com -> Add new project -> import saas-gastro.
- Framework: Next.js (auto-detectado).
- Root directory: /
- Environment variables: pegar TODO el .env.local en Settings -> Environment Variables.
- CAMBIAR NEXT_PUBLIC_APP_URL a la URL de Vercel (ej. saas-gastro.vercel.app).
- Deploy.

16.2 Configurar dominio

Vercel te da saas-gastro.vercel.app gratis. Si queres dominio propio, compra .com.py en NIC.PY (~30 USD/ano) o .com en Namecheap (~12 USD/ano), apuntas DNS a Vercel.

16.3 Configurar deploys preview

Por defecto Vercel hace un deploy de cada PR. Usalo: cada feature en su rama, te regenera una URL saas-gastro-git-feat-x.vercel.app para testear sin tocar produccion.

16.4 Health check post-deploy

- Abrir la URL pelada -> deberia ir a /admin (login).
- Loguear con tu cuenta admin.
- Crear una mesa, generar QR, escanearlo desde el celular en otra red.
- Si todo bien: confirmar pedido. Abrir /kds en otra pestana y ver que aparece en realtime.

16.5 Commit y push

```
git add .
git commit -m "feat: deploy v1.0 production-ready"
git push origin main
```

Paso 17 - Test end-to-end con QR real

Tiempo: 1 hora. La prueba final.

17.1 Setup

- Imprimir 3-5 QRs (de mesas activas).
- Pegarlos en mesas reales (tu casa, oficina, lo que sea).
- Tener 2-3 dispositivos: celular del 'cliente', tablet de 'cocina', laptop de 'admin'.

17.2 Checklist E2E

- [ ] Escanear QR con celular -> abre el menu de la mesa correcta.
- [ ] Agregar 3 productos al carrito.
- [ ] Ir al checkout, completar nombre + tel + RUC.
- [ ] Elegir 'efectivo' y confirmar.
- [ ] Pantalla de seguimiento aparece con estado 'pendiente'.
- [ ] Abrir /kds en tablet -> el pedido aparece en cocina/bar segun station.
- [ ] Marcar todos los items como 'listo' uno por uno.
- [ ] Ver que el cliente recibe update 'tu pedido esta listo' (sin refresh).
- [ ] Marcar order como entregado en KDS.
- [ ] Marcar como cobrado en caja.
- [ ] Cliente ve el pedido como cerrado en su pantalla.
- [ ] Volver a /admin -> el stock del producto bajo correctamente.
- [ ] Volver a /admin/stats -> el pedido cuenta en 'pedidos hoy' y 'facturado hoy'.
- [ ] Repetir desde otro celular para confirmar que es repetible.

17.3 Si algo falla

- Vercel logs: dashboard -> Deployments -> seleccionar -> Logs.
- Supabase logs: dashboard -> Logs -> filtros por table o por error.
- Browser console del cliente: F12 -> Console / Network.
- Si Realtime no funciona: chequear que las tablas esten en Replication ON.

17.4 Que tenes ahora

Una demo funcional, en internet, con QR escaneable, tickets en realtime, panel admin completo, stock que se descuenta solo, stats basicos. Cero pesos al mes (excluyendo Claude Code). Eso es lo que vas a mostrarles a tus 3 socios y a los duenos de restaurante en las 15 entrevistas.

Apendice A - Estructura final de carpetas

```
saas-gastro/
.env.local # gitignored
.gitignore
next.config.js
package.json
tsconfig.json
tailwind.config.ts
README.md
CONTEXT.md # quien soy, que pretendo
PROGRESS.md # log cronologico
DECISIONS.md # ADRs
/docs
/handoff/
2026-04-28-v1-bootstrap.md
/supabase
/migrations/ # archivos SQL versionados
seed.sql
/src
/app
/(admin)/admin
page.tsx # dashboard
/menu
/mesas
/qrs
/stock
/perfil
/stats
layout.tsx
/(cliente)/m/[slug]
page.tsx
/carrito
/checkout
/seguimiento/[orderId]
/(kds)/kds
page.tsx
/login
page.tsx
/api
/webhooks
(placeholders Bancard fase 2)
layout.tsx
page.tsx # landing publica simple
globals.css
/components
/admin
/cliente
/kds
/ui # shadcn
/hooks
use-carrito.ts
use-realtime-orders.ts
/lib
/supabase
client.ts
server.ts
/schemas
checkout.ts
producto.ts
mesa.ts
/types
database.ts # generado por supabase gen types
```

```
utils.ts
/middleware.ts # protege rutas /admin/*
```

Apendice B - Prompts utiles para Claude Code

Pegale estos prompts cuando estes en cada paso. No pidas 'haceme la app entera': pedile feature por feature.

B.1 Para arrancar (despues del Paso 2)

```
Sos un dev senior Next.js 14 + Supabase. Estamos haciendo un SaaS gastronomico.
Lee CONTEXT.md, PROGRESS.md y DECISIONS.md antes de cualquier cambio. Despues de cada
feature terminada, agrega un archivo en /docs/handoff/[fecha]-[feature].md
describiendo:
- Que se hizo
- Que archivos se tocaron
- Que decisiones se tomaron y por que
- Que falta por hacer / TODOs encontrados
- Como testear lo hecho

Confirmame que entendiste y leiste los 3 archivos antes de avanzar.
```

B.2 Para implementar una pagina

```
Implementa la pagina /admin/mesas siguiendo lo descrito en
/docs/v1-paso-a-paso.pdf seccion 6.

Stack: Next.js 14 App Router (server components + server actions),
shadcn/ui, react-hook-form + zod, Supabase ssr.

Restricciones:
- No uses 'use client' a menos que sea estrictamente necesario.
- Las mutaciones siempre por server actions (NO por API routes).
- Toast de exito/error con sonner.
- Siempre validar con zod ANTES de tocar la DB.
- Usar revalidatePath en server actions.

Despues de implementar, hace commit con mensaje convencional
(feat:, fix:, chore:) y agrega el handoff.
```

B.3 Para resolver un bug

```
Bug: cuando un cliente confirma un pedido, el item no aparece en /kds aunque
Realtime esta activo en order_items.

Antes de proponer cambios:
1. Leeme los archivos involucrados (KdsView, kds-view subscription).
2. Decime tu hipotesis principal y 2 alternativas.
3. Como verificarlas (que mirar en la consola del browser y en Supabase logs).

No edites nada hasta confirmar la causa.
```

B.4 Para revisar codigo

```
Revisa /src/app/(cliente)/m/[slug]/checkout/actions.ts.
Foco:
- Race conditions al insertar order + items.
- Manejo de errores (que pasa si el segundo INSERT falla).
- Validacion zod (esta antes de las queries?).
- Inputs no sanitizados (RUC, telefono).

Reporta hallazgos en formato:
Severidad / Linea / Problema / Sugerencia.
No edites nada todavia.
```

B.5 Antes de un deploy

```
Pre-deploy check de v1.0:
1. Corre pnpm build y reportame errores.
2. Corre pnpm lint y reportame warnings.
3. Confirma que .env.example esta actualizado.
4. Confirma que no hay console.log olvidados en src/.
5. Confirma que las migrations en /supabase/migrations estan aplicadas en el proyecto live.
```

Apendice C - Errores comunes y soluciones

Sintoma	Causa probable	Solucion
pnpm dev no arranca	Falta .env.local o keys mal copiadas.	Revisar las 3 vars NEXT_PUBLIC_* y service_role_key.
'Module not found: @supabase/ssr'	Falto pnpm install.	pnpm add @supabase/supabase-js @supabase/ssr
RLS violado al crear order	Politicas FOR INSERT no permiten anon.	Revisar politica 'anyone crea order' y rerun.
Realtime no actualiza KDS	Tabla no esta en Replication.	Supabase Dashboard -> Database -> Replication -> activar order_items.
Foto de producto no carga	Bucket no es publico o policy de SELECT falta.	Marcar bucket como public + politica 'public read productos'.
Build OK pero crash en Vercel	Falta env var en Vercel (solo en local).	Project Settings -> Environment Variables -> agregar todas.
Cookies de Auth no persisten	Importaste createClient incorrecto (browser en server).	client.ts para client components, server.ts para server.
'cookies() should be awaited'	Next.js 15+ requiere await cookies().	Usar `const cookieStore = await cookies()`.
Stock no se descuenta	Trigger trg_descontar_stock no creado.	Re-correr el SQL del Paso 3 que crea la funcion y trigger.
Total del order da 0	Trigger trg_recalcular_total no creado o con error.	Verificar SELECT FROM order_items con WHERE order_id correcto.
QR no abre el menu	URL del QR usa NEXT_PUBLIC_APP_URL viejo.	Actualizar env var en Vercel y redeploy. Regenerar QRs.
Cliente ve order de otro cliente	Politicas 'anyone lee orders' permiten todo.	v1.0 conocido. En v2 restringir por session token o cookie.
Realtime se cae despues de 1h	Free tier Supabase pausea proyectos inactivos.	Pasar a Pro (USD 25) o pingear con cron.
Build fail por 'Cannot find module'	Path alias mal en tsconfig.	Verificar baseUrl y paths en tsconfig.json.

Cuando salga un error que no este aca, agregalo a /docs/handoff/[fecha]-error-X.md con: sintoma, log completo, causa real, solucion. En 2 meses vas a tener tu propia documentacion sobre el sistema, mas valiosa que cualquier curso.

Fin del PDF v1.0. Cuando termines los 17 pasos y tengas el sistema funcionando, te paso v2.0: integracion Bancard real, SIFEN real, multi-tenant, panel super admin (el del PDF anterior), bolsa de trabajo, comunidad, tienda de proveedores. Pero antes de eso: 15 entrevistas a restaurantes con la demo en mano.